

ISO SECURE

Plataforma de Cumplimiento ISO 27001

Etapa 3: Codificación e Integración

Actividad Formativa 3 - 5 %

Agustín Peralta — Universidad de San Buenaventura

Entrega: 11 de Mayo de 2026

Resumen

Este documento consolida las prácticas de codificación segura aplicadas en el proyecto ISO SECURE, el flujo de integración continua, la estrategia de control de versiones, los análisis estáticos y de dependencias, y las pruebas unitarias/integración que sustentan la fase de construcción del SDLC seguro. Se mapean las prácticas a OWASP ASVS 4.0 y a los controles del Anexo A de ISO/IEC 27001:2022 (cláusula A.8 — controles tecnológicos).

Índice

1. Introducción	2
2. Prácticas de Codificación Segura	2
2.1. Mapeo a OWASP Top 10:2021	2
2.2. Reglas internas de código seguro	3
2.3. Ejemplos de implementación	3
3. Control de Versiones y Flujo Git	4
3.1. Estrategia de ramas	4
3.2. Convención de commits	4
3.3. Política de Pull Requests	5
4. Pipeline de Integración Continua	5
4.1. Etapas del pipeline	5
4.2. Configuración de ejemplo (GitHub Actions)	5
5. Análisis Estático (SAST) y de Dependencias (SCA)	6
5.1. Herramientas seleccionadas	6
5.2. Hallazgos típicos y política de fix	6
5.3. Pre-commit hooks	6

6. Estrategia de Pruebas	7
6.1. Pirámide de pruebas	7
6.2. Tipos de prueba	7
6.3. Ejemplo: prueba de RBAC multi-tenant	7
6.4. Cobertura mínima exigida	8
7. Integración de Componentes	8
7.1. Stack de integración	8
7.2. Migración determinística	8
7.3. Entornos	9
8. Gestión de Secretos	9
9. Trazabilidad: Diseño → Implementación	9
10. Conclusiones	10

1. Introducción

La Etapa 3 traduce las decisiones de arquitectura y diseño (Etapa 2) en código de producción siguiendo los principios del **Secure Software Development Lifecycle (SS-DLC)** de OWASP SAMM y NIST SP 800-218. Esta fase es donde la mayor parte de las vulnerabilidades se introducen — y donde se previenen.

Objetivos de la fase:

- Codificar respetando las guías OWASP Secure Coding Practices.
- Establecer un flujo de Git con revisión obligatoria por pares.
- Automatizar análisis estático (SAST) y de dependencias (SCA) en CI.
- Garantizar cobertura mínima de pruebas (>70 % líneas críticas).
- Integrar los componentes (backend, frontend, base de datos) en entornos reproducibles.

2. Prácticas de Codificación Segura

2.1. Mapeo a OWASP Top 10:2021

ID	Riesgo OWASP	Práctica aplicada en ISO SECURE
A01	Broken Access Control	<code>require_role()</code> factory + <code>helper_empresa_filter</code> en todos los routers
A02	Cryptographic Failures	TLS 1.3 obligatorio; secretos vía variables de entorno; tokens JWT firmados RS256
A03	Injection	ORM SQLAlchemy parametrizado; validación Pydantic v2 estricta; sin string concatenation en queries
A04	Insecure Design	Modelado de amenazas previo (Etapa 2); revisión arquitectónica antes de codificar
A05	Security Misconfiguration	Headers CSP, HSTS, X-Content-Type-Options en Nginx; <code>DEBUG=False</code> en producción
A06	Vulnerable Components	Dependabot + <code>pip-audit</code> + <code>npm audit</code> en CI
A07	Auth Failures	Supabase Auth gestiona contraseñas, MFA y bloqueos; tokens de corta vida
A08	Data Integrity Failures	Hash de integridad en releases; verificación de firmas en webhooks n8n
A09	Logging Failures	Logs estructurados JSON; mask de campos sensibles; envío a sink externo
A10	SSRF	Whitelist de URLs salientes; sin proxy abierto; n8n con allowlist de hosts

2.2. Reglas internas de código seguro

Backend (Python):

1. Nunca aceptar `empresa_id` desde el body del request; inferirlo del usuario autenticado.
2. Toda función pública en `routers/` declara dependencias `Depends()` explícitas (sesión + usuario).
3. Schemas Pydantic separados por dirección: `XCreate`, `XUpdate`, `XResponse` (no reutilizar el modelo ORM como schema HTTP).
4. Errores genéricos hacia el cliente en producción; detalles sólo en logs internos.
5. `require_role()` declarado al nivel del endpoint, no dentro de la función.

Frontend (React):

1. Tokens en `localStorage` encapsulados en un adapter; no `innerHTML` ni `dangerouslySetInnerHTML`.
2. Validación en cliente **además** de la del servidor; nunca como reemplazo.
3. Interceptor Axios añade el JWT y captura 401 para redirigir al login.
4. Variables de entorno con prefijo `VITE_` y nunca incluir secretos en el bundle.
5. Render condicional con guards de rol antes de renderizar acciones sensibles.

2.3. Ejemplos de implementación

Listado seguro de incidentes (backend):

```
1 @router.get("/incidents", response_model=List[IncidentResponse
2     ])
3 async def list_incidents(
4     db: AsyncSession = Depends(get_db),
5     user: UserProfile = Depends(get_current_user),
6 ):
7     # empresa_id se infiere del usuario, NUNCA del request
8     empresa_id = _empresa_filter(user)
9     return await incident_service.list_for(db, empresa_id=
10         empresa_id)
```

Guard de rol como factory (backend):

```
1 def require_role(allowed: list[str]):
2     async def _guard(user: UserProfile = Depends(
3         get_current_user)):
4         if user.role not in allowed:
5             raise HTTPException(403, detail="forbidden")
6         return user
7     return _guard
8
9 # Uso:
10 @router.put("/auth/role")
```

```

10 async def change_role(
11     payload: ChangeRoleSchema,
12     admin: UserProfile = Depends(require_role(["admin"])),
13 ):
14     ...

```

Interceptor Axios (frontend):

```

1 import axios from "axios";
2
3 const api = axios.create({
4     baseURL: import.meta.env.VITE_API_URL + "/api/v1",
5 });
6
7 api.interceptors.request.use((config) => {
8     const token = sessionStorage.getToken();
9     if (token) config.headers.Authorization = `Bearer ${token}`;
10    return config;
11 });
12
13 api.interceptors.response.use(
14     (r) => r,
15     (err) => {
16         if (err.response?.status === 401) sessionStorage.logout();
17         return Promise.reject(err);
18     }
19 );

```

3. Control de Versiones y Flujo Git

3.1. Estrategia de ramas

Se adopta una variante de **Trunk-Based Development** adaptada al tamaño del equipo:

- **main**: rama protegida; sólo merges vía PR aprobado y CI verde.
- **feature/<slug>**: ramas de corta vida (<3 días); se mergean a main y se eliminan.
- **hotfix/<slug>**: parches urgentes desde main, mergeados con prioridad.

3.2. Convención de commits

Se usa **Conventional Commits** para habilitar changelog automático:

```

1 <tipo>(<scope>): <descripcion>
2
3 tipos: feat, fix, docs, refactor, test, chore, perf, security

```

Ejemplos reales del repositorio:

- feat(etapa-1): Análisis e Ingeniería de Requerimientos - PDF entregable

- feat: agregar `_headers` con CSP y cabeceras de seguridad
- docs: agregar metodología PASTA + STRIDE para modelamiento de amenazas

3.3. Política de Pull Requests

1. Toda PR debe pasar el pipeline de CI (lint, tests, SAST, SCA).
2. Al menos un revisor humano aprueba antes del merge.
3. No se mergean PRs con conflictos sin resolver.
4. Squash merge para mantener historial lineal.
5. Los commits a main quedan bloqueados sin PR (branch protection).

4. Pipeline de Integración Continua

4.1. Etapas del pipeline

Etapa	Herramienta	Falla si
Lint Python	ruff	errores E/F
Lint JS/JSX	ESLint + prettier	cualquier error
SAST Python	bandit	severidad $\geq$ medium
SAST JS	semgrep	regla bloqueante
SCA Python	pip-audit	CVE crítica/alta
SCA JS	npm audit + Snyk	CVE crítica
Tests unitarios	pytest + jest	cualquier fallo
Cobertura	coverage.py + jest --coverage	$< 70\%$ backend
Build Docker	docker build	error de build
Scan imagen	Trivy	CVE crítica

4.2. Configuración de ejemplo (GitHub Actions)

```

1  name: ci
2  on: [push, pull_request]
3  jobs:
4    backend:
5      runs-on: ubuntu-latest
6      steps:
7        - uses: actions/checkout@v4
8        - uses: actions/setup-python@v5
9          with: { python-version: "3.13" }
10       - run: pip install -r requirements.txt -r requirements-
           dev.txt
11       - run: ruff check .
12       - run: bandit -r app -lll
13       - run: pip-audit --strict
14       - run: pytest --cov=app --cov-fail-under=70
15  frontend:

```

```

16     runs-on: ubuntu-latest
17     steps:
18     - uses: actions/checkout@v4
19     - uses: actions/setup-node@v4
20       with: { node-version: "20" }
21     - working-directory: frontend
22       run: |
23         npm ci
24         npm run lint
25         npm audit --audit-level=high
26         npm test -- --coverage
27         npm run build

```

5. Análisis Estático (SAST) y de Dependencias (SCA)

5.1. Herramientas seleccionadas

Herramienta	Tipo	Cobertura
ruff	Linter	estilo + bugs comunes Python
bandit	SAST	vulnerabilidades comunes Python
semgrep	SAST multilenguaje	reglas custom + community rules
pip-audit	SCA Python	CVE en dependencias
npm audit	SCA Node	CVE registry
Trivy	Scanner contenedores	CVE en imagen Docker
Dependabot	Actualizaciones	PRs automáticas

5.2. Hallazgos típicos y política de fix

Hallazgo	Acción	SLA
CVE crítica (CVSS ≥ 9.0)	Fix obligatorio antes del merge	0 días
CVE alta (7.0-8.9)	Fix antes del próximo release	7 días
CVE media (4.0-6.9)	Backlog priorizado	30 días
Bandit MEDIUM/HIGH	Análisis manual + fix o supresión justificada	14 días
ruff/eslint	Auto-fix vía pre-commit hook	0 días

5.3. Pre-commit hooks

Se usa `pre-commit` con la siguiente configuración mínima para que los problemas se detecten antes del push:

```

1 # .pre-commit-config.yaml
2 repos:
3   - repo: https://github.com/astral-sh/ruff-pre-commit
4     rev: v0.5.0

```

```

5     hooks: [{ id: ruff, args: [--fix] }, { id: ruff-format }]
6 - repo: https://github.com/PyCQA/bandit
7     rev: 1.7.9
8     hooks: [{ id: bandit, args: [-r, app] }]
9 - repo: local
10    hooks:
11      - id: no-secrets
12        name: detect-secrets
13        entry: detect-secrets-hook
14        language: system

```

6. Estrategia de Pruebas

6.1. Pirámide de pruebas

- **Base (70 %):** Unitarias — services y utils aislados.
- **Medio (20 %):** Integración — routers contra DB de prueba.
- **Top (10 %):** End-to-end — flujos críticos vía Playwright.

6.2. Tipos de prueba

Tipo	Foco	Herramienta
Unitaria	Lógica pura de servicios, validaciones, helpers	pytest
Integración	Router + DB + Auth real	pytest-asyncio + httpx
Contrato	Compatibilidad de schemas Pydantic	schemathesis
Seguridad	RBAC, multi-tenancy, JWT inválido	pytest + escenarios
E2E	Login + listado + creación de incidente	Playwright
Carga	Throughput de /dashboard/summary	Locust o k6

6.3. Ejemplo: prueba de RBAC multi-tenant

```

1  @pytest.mark.asyncio
2  async def test_supervisor_solo_ve_su_empresa(client, db,
3         empresas, users):
4      # Arrange: 2 empresas con 1 incidente cada una
5      sup_token = await login(users.supervisor_emp_a)
6
7      # Act
8      r = await client.get(
9          "/api/v1/incidents",
10         headers={"Authorization": f"Bearer {sup_token}"},
11     )
12
13     # Assert
14     assert r.status_code == 200

```



```
14     ids = {i["empresa_id"] for i in r.json()}
15     assert ids == {str(empresas.a.id)}, "Filtro empresa_id
        fall "
```

6.4. Cobertura mínima exigida

- Backend global: $\geq 70\%$ líneas.
- Módulos críticos (auth, multi-tenant filter): $\geq 90\%$.
- Frontend: $\geq 60\%$ líneas; 80% en componentes con lógica de negocio.

7. Integración de Componentes

7.1. Stack de integración

Se usa **Docker Compose** para orquestar el entorno de desarrollo y staging:

```
1  # docker-compose.yml (extracto real del repo)
2  version: "3.8"
3  services:
4      api:
5          build: .
6          ports: ["8000:8000"]
7          environment:
8              - DATABASE_URL=${DATABASE_URL}
9              - SUPABASE_URL=${SUPABASE_URL}
10             - SUPABASE_ANON_KEY=${SUPABASE_ANON_KEY}
11          depends_on: [postgres]
12      postgres:
13          image: postgres:15-alpine
14          ports: ["5432:5432"]
15          environment:
16              - POSTGRES_PASSWORD=devpass
17          volumes:
18              - pgdata:/var/lib/postgresql/data
19  volumes:
20      pgdata:
```

7.2. Migración determinística

El script `migrate.py` ejecuta los pasos de esquema en orden idempotente, evitando que dos despliegues lleguen a estados divergentes. Incluye:

1. Rename de enums legados (`partial` $\rightarrow$ `en_proceso`).
2. Creación de tipos enum nuevos.
3. Creación de tablas (`empresas`, `user_profiles`, `cursos`, etc.).
4. Seed de cursos iniciales con URLs validadas.

5. Adición de FK `empresa_id` a `incidents`, `controls`, `risk_levels`.
6. Seed automático de los 93 controles ISO por sector detectado.

7.3. Entornos

Entorno	Propósito	Datos
dev local	Desarrollo personal	PostgreSQL en Docker + seed
staging	QA, demos, UAT	Supabase staging + datos sintéticos
producción	Usuarios reales	Supabase prod + backups diarios

8. Gestión de Secretos

- `.env` en `.gitignore`; nunca commiteado.
- `.env.example` en el repo con placeholders.
- Producción: secretos en el gestor del proveedor (Supabase Vault, GitHub Actions Secrets).
- Rotación de la `SUPABASE_ANON_KEY` y `SERVICE_ROLE_KEY` cada 90 días.
- Hook `detect-secrets` en pre-commit bloquea pushes con tokens accidentales.

9. Trazabilidad: Diseño → Implementación

Decisión Etapa 2	Implementación	Validación
Filtro <code>empresa_id</code> server-side	<code>_empresa_filter</code> en routers	Test <code>test_multitenant_isolation</code>
JWT validado vs Supabase	<code>auth.py::get_current_user</code>	Test <code>test_jwt_invalid_rejected</code>
RBAC factory	<code>require_role()</code>	Test <code>test_supervisor_no_puede_borrar</code>
ORM parametrizado	SQLAlchemy 2.0 async	SAST bandit sin findings
Migraciones idempotentes	<code>migrate.py</code> con guards IF NOT EXISTS	CI corre migrate dos veces
Logs sin secretos	Mask en <code>logging.Filter</code>	Grep en logs de staging

10. Conclusiones

La Etapa 3 materializa la arquitectura en código siguiendo prácticas de codificación segura mapeadas a OWASP Top 10:2021 y a OWASP ASVS 4.0. Los puntos clave del cierre de esta fase son:

1. Pipeline CI con SAST, SCA y tests es **bloqueante**: ningún merge sin CI verde.
2. Cobertura mínima 70 % global y 90 % en módulos críticos (auth, multi-tenant).
3. Convención Conventional Commits + branch protection en **main**.
4. Pre-commit hooks llevan el shift-left al máximo: el desarrollador ve los problemas antes del push.
5. Cada decisión arquitectónica de la Etapa 2 tiene su contrapartida verificable en código y prueba.

La fase de codificación no termina con el primer release: las prácticas, hooks y políticas descritas son parte del ciclo de mejora continua exigido por ISO 27001:2022 (cláusulas 9 y 10).

Referencias

- OWASP Top 10:2021.
- OWASP Application Security Verification Standard (ASVS) 4.0.3.
- OWASP Secure Coding Practices Quick Reference Guide v2.0.
- NIST SP 800-218: Secure Software Development Framework (SSDF).
- ISO/IEC 27001:2022 — Anexo A.8 (Controles tecnológicos).
- Conventional Commits 1.0.0 — <https://www.conventionalcommits.org/>.